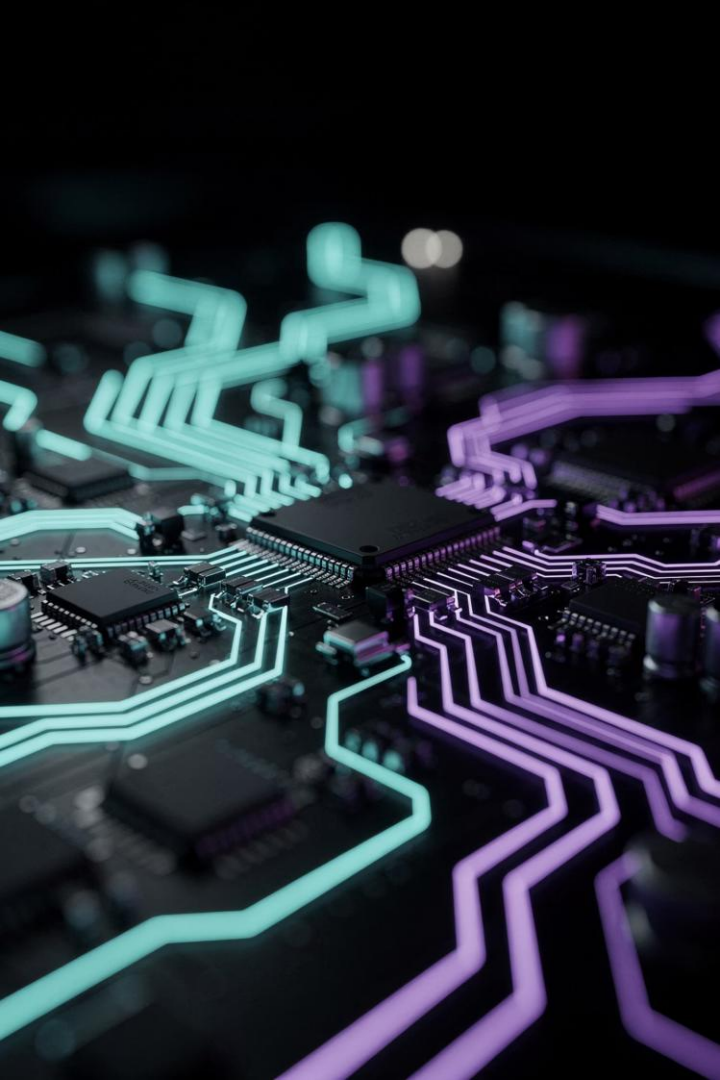




TinyML: Machine Learning at the Edge – Part 2

A Complete Lecture on TinyML

Undergraduate Engineering – Neural Networks & Deep Learning

SECTION 10

Data Engineering for TinyML

Every high-performing TinyML model begins with rigorous data engineering. The quality of edge intelligence is fundamentally bounded by the quality of the data pipeline — from raw collection to curated, balanced, and ethically governed datasets ready for embedded deployment.

Collect

Capture representative, real-world sensor data

Clean

Remove noise, artifacts, and corrupt samples

Label

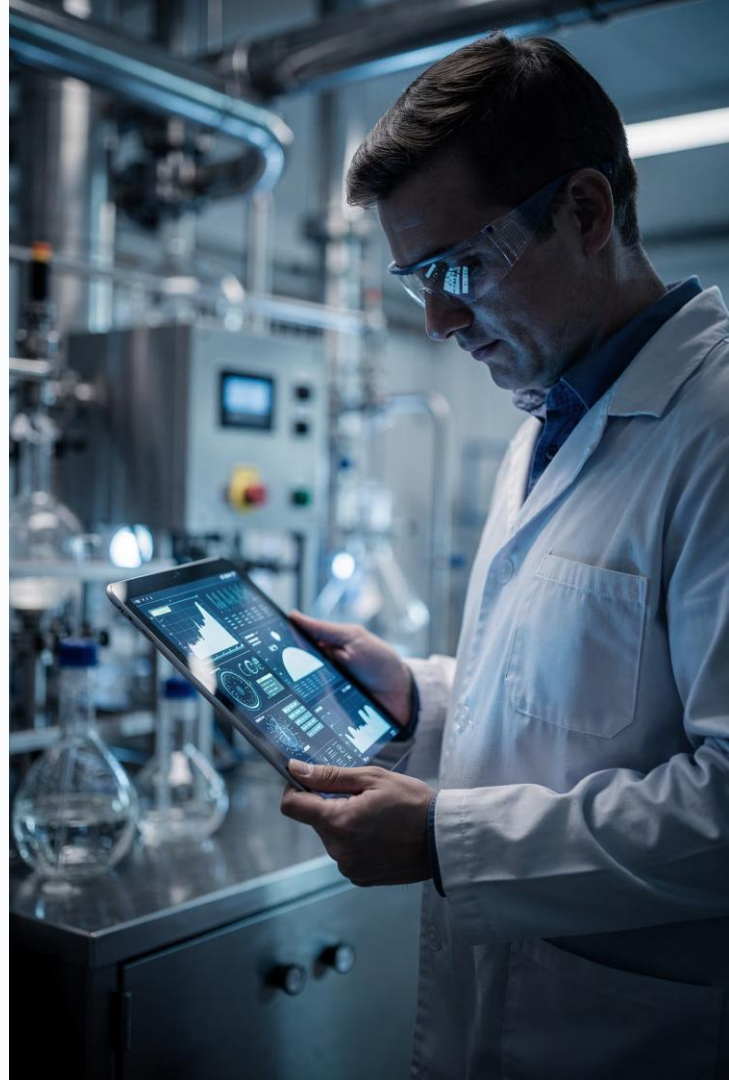
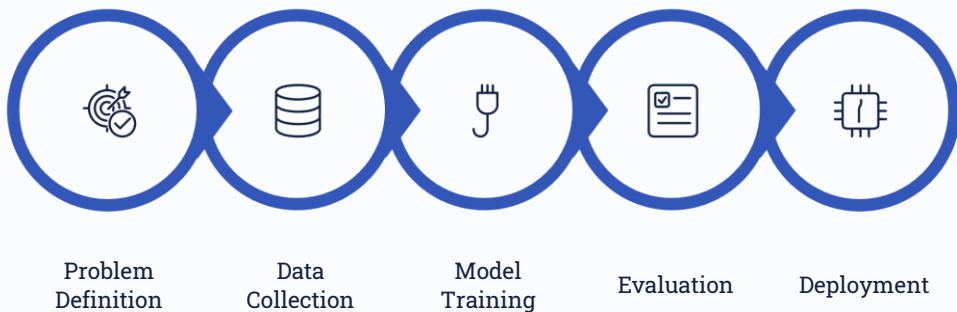
Annotate with accurate, consistent ground truth

Curate

Balance, augment, and refine for edge constraints

The AI Lifecycle

TinyML systems follow a closed-loop AI lifecycle. Unlike cloud ML, every phase is influenced by the constraints of embedded hardware — memory budgets, latency targets, and power envelopes shape decisions from the very first data collection step.



Data Collection for TinyML

What to Capture

- Real deployment environment samples
- Edge-case and negative examples
- Diverse sensor modalities (audio, IMU, image)
- Balanced class distribution from the start

Collection Strategies

- On-device logging directly from target MCU
- Crowd-sourced data via mobile devices
- Synthetic generation for rare events
- Open datasets as a baseline (then fine-tune)



Data collected in a controlled lab may fail silently in real deployment conditions.

Data Labeling

Labeling is the most human-intensive step of the pipeline. For TinyML, mislabeled data has an outsized impact – small datasets amplify every annotation error, potentially corrupting entire classes in a model trained on hundreds rather than millions of samples.



Manual Annotation

Human experts review and tag each sample. High accuracy but slow and expensive at scale.



Programmatic Labeling

Weak supervision rules auto-label large volumes. Fast but requires careful validation.



Label Verification

Inter-annotator agreement scores and confusion audits catch systematic mislabeling early.

Data Curation

Raw labeled data is never directly training-ready. Curation is the deliberate process of shaping the dataset so that it reflects the real operating distribution of the deployed model — with clean, balanced, and representative samples.

Deduplication

Remove near-duplicate samples that cause overfitting to specific recording sessions.

Class Balancing

Oversample minority classes or undersample majorities to prevent bias toward frequent events.

Quality Filtering

Drop clipped audio, motion-blurred images, or corrupt sensor readings using automated heuristics.

Split Discipline

Ensure train/val/test splits are temporally or speaker-disjoint to prevent data leakage.

Dataset Refinement & Sustainment

Refinement (Before Launch)

- Active learning: query the hardest examples
- Error analysis on validation confusion matrix
- Targeted re-collection for underperforming classes
- Human-in-the-loop relabeling cycles

Sustainment (Post-Launch)

- Monitor live data distribution drift
- Retrain periodically with new field samples
- Version datasets alongside model checkpoints
- Maintain an evergreen annotation pipeline



Deployed TinyML models can silently degrade as real-world conditions shift. Sustainment is not optional.

Data Augmentation

Augmentation artificially expands training datasets by applying label-preserving transformations. For TinyML, augmentation is especially powerful because field-collected datasets are often small – augmentation regularizes the model and improves robustness to deployment variability.

Audio

Time-shift, pitch-shift, noise injection, speed perturbation, SpecAugment (frequency masking)

Image

Flips, crops, brightness/contrast jitter, random erasing, Mixup, CutMix

IMU / Sensor

Rotation invariance, Gaussian noise, sensor dropout simulation, window jitter

Dataset Bias

Dataset bias occurs when training data does not faithfully represent the deployment population. In TinyML, biased models can fail silently — a keyword spotter trained on adult American English will underperform for children or non-native speakers with no error messages on the MCU.

→ Collection Bias

Over-representing one demographic, accent, or environment at recording time.

→ Labeling Bias

Annotators systematically disagreeing or applying subjective labels inconsistently.

→ Deployment Bias

Model evaluated in lab but deployed in a fundamentally different real-world context.



Licensing & Data Ethics



License Compliance

Understand CC-BY, CC-BY-SA, and restrictive licenses before incorporating open datasets into commercial TinyML products.



Consent & Privacy

Audio and image data often contains PII. Obtain informed consent; anonymize speaker identities before distribution.



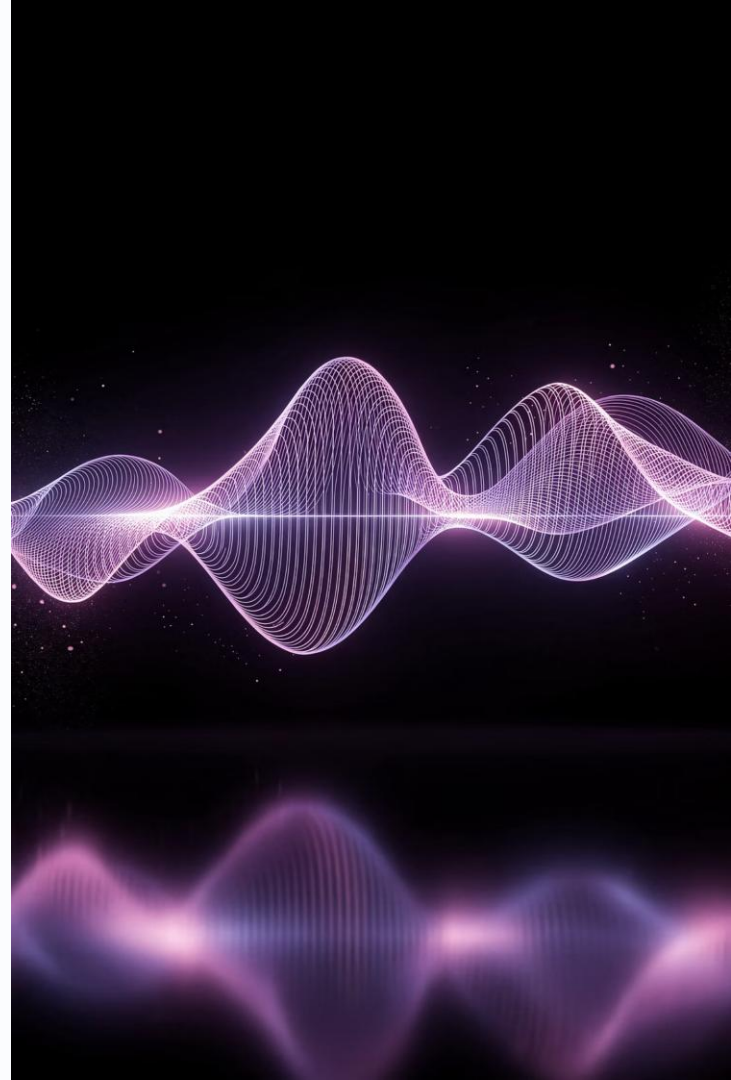
Fairness Obligations

Document protected-attribute performance breakdowns. Publish a data card alongside every released dataset.

SECTION 11

Keyword Spotting

Detecting spoken trigger words on microcontrollers — always-on, milliwatt-level audio intelligence.



What Is Keyword Spotting?

Definition

Keyword Spotting (KWS) is the task of detecting a small set of predefined words or phrases in a continuous audio stream — in real time, on an always-on embedded processor — without sending audio to the cloud.

Common Applications

- "Hey Siri" / "OK Google" wake-word detection
- Industrial voice command interfaces
- Accessibility devices and hearing aids
- Smart home appliances and wearables

Why KWS Is Ideal for TinyML



Ultra-Low Power

A microphone and MCU consuming under 1 mW can run KWS continuously for months on a coin cell battery.



Privacy Preserving

Audio never leaves the device. Only a binary decision (keyword detected / not detected) is transmitted.



Small Model Footprint

DS-CNN models achieve >90% accuracy in under 80 KB — well within Cortex-M4 flash budgets.



Low Latency

Inference over a 1-second audio window completes in <10 ms on modern MCUs, enabling real-time response.

Google Speech Commands Dataset

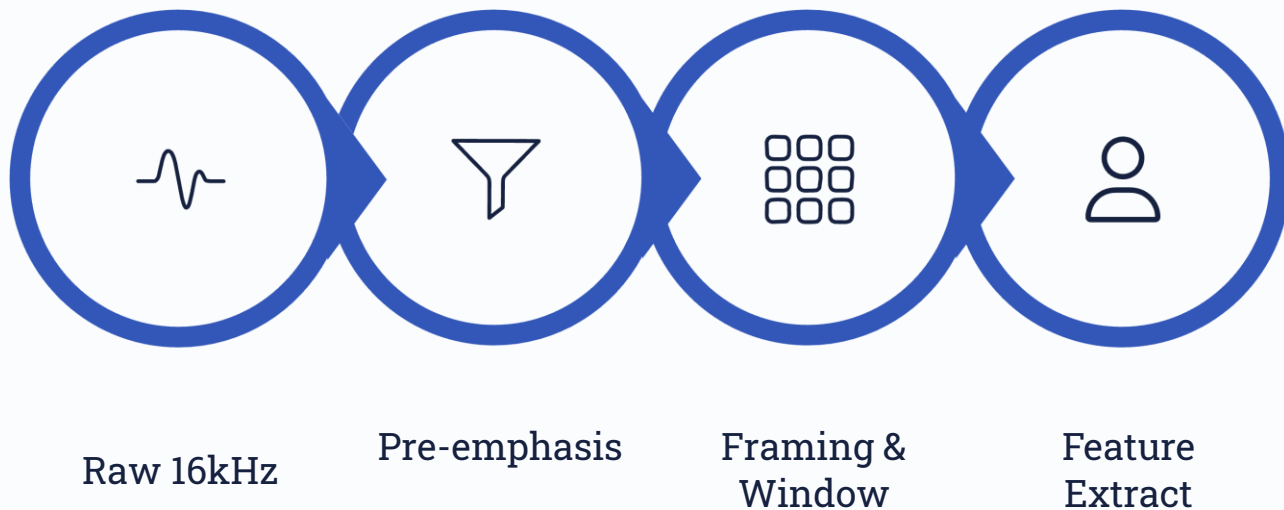
Dataset Overview

- 105,829 one-second WAV clips at 16 kHz
- 35 keyword classes (e.g., "yes", "no", "stop", "go")
- 2,618 unique speakers across diverse demographics
- Standard splits: 80% train / 10% val / 10% test

TinyML Usage

- Benchmark standard for KWS research papers
- 10-class subset most common for MCU deployment
- Background noise clips included for augmentation
- CC-BY 4.0 license — free for commercial use

Audio Preprocessing Pipeline



Raw audio cannot be fed directly into a CNN. Preprocessing converts the time-domain waveform into a compact 2D feature map — essentially a spectrogram image — which the neural network treats as a visual pattern recognition task. Each 1-second clip becomes a 49×40 input tensor.

Short-Time Fourier Transform (STFT)

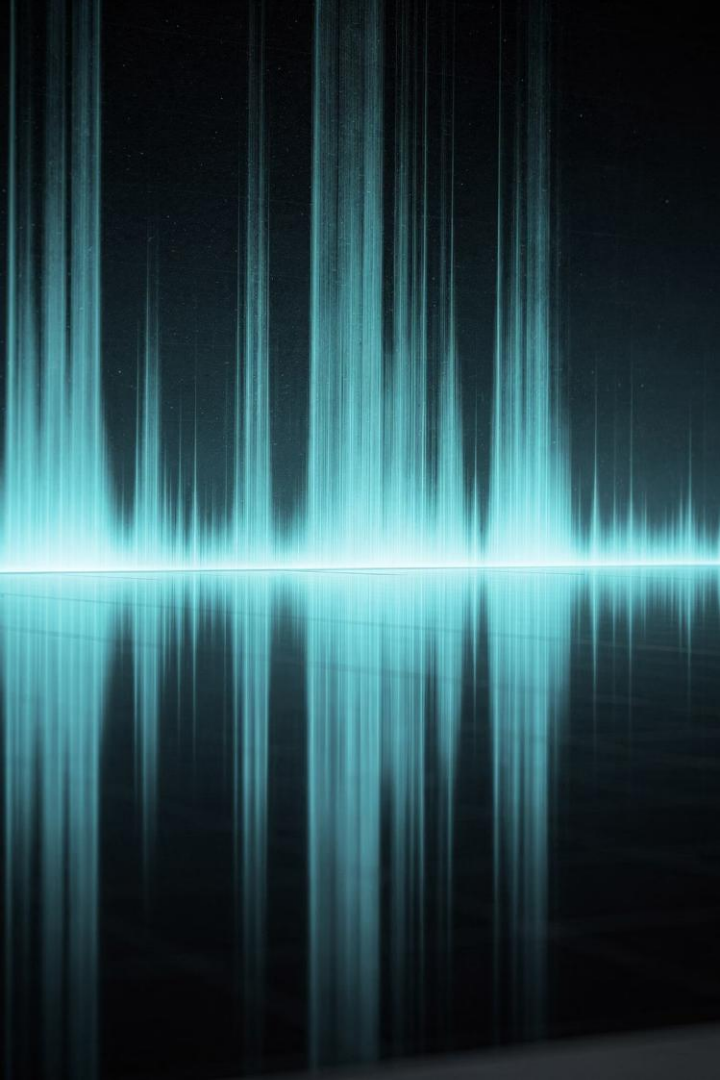
Concept

The STFT slides a short analysis window (typically 25 ms) across the audio signal with a hop size (10 ms overlap), computing the FFT of each window. The result is a time-frequency matrix: the **spectrogram**.

Key Parameters

- **Window length:** 25 ms $\rightarrow$ 400 samples at 16 kHz
- **Hop length:** 10 ms $\rightarrow$ 160 samples stride
- **FFT bins:** 512 $\rightarrow$ 257 unique frequency bins
- **Output:** 49 frames $\times$ 257 bins per 1-second clip

 The STFT magnitude squared gives the power spectrogram. Phase information is discarded for classification tasks.



Mel Spectrogram

The Mel scale warps linear frequency bins to match human auditory perception — closely spaced at low frequencies, wider at high frequencies. Mel spectrograms compress the 257-bin STFT to 40 Mel filterbank channels, producing a 49×40 tensor that emphasizes perceptually relevant phonemic features. This compact representation is the standard input format for KWS CNNs in TinyML deployments.

DS-CNN Architecture

Depthwise Separable CNN (DS-CNN) is the dominant KWS backbone for TinyML. It factorizes standard 3×3 convolutions into a **depthwise** (per-channel spatial filter) and a **pointwise** (1×1 cross-channel mixing) convolution, reducing multiply-accumulate operations by $\sim 8-9\times$ while maintaining accuracy.



Quantization & Distillation in KWS

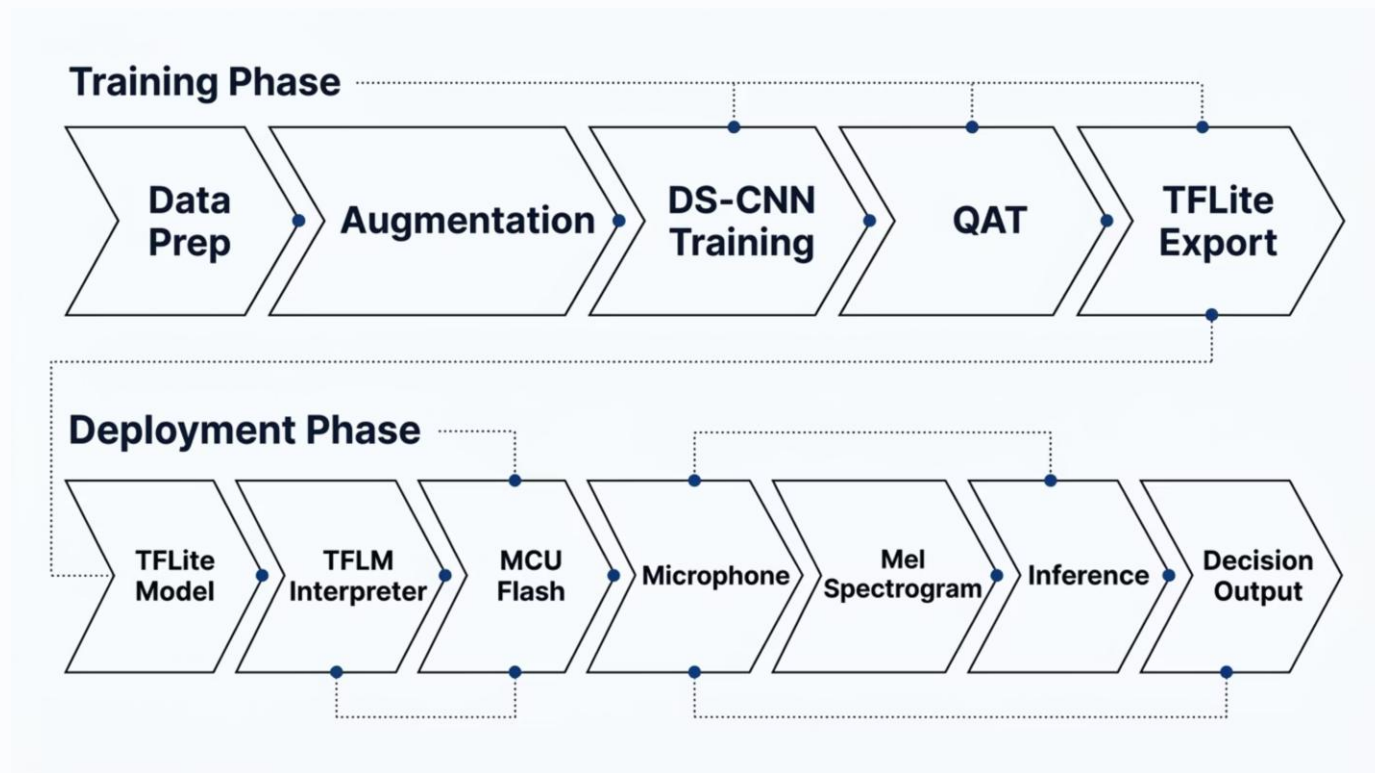
INT8 Quantization Effects

- Model size: ~80 KB → ~20 KB (4× reduction)
- Inference: float32 MACs → INT8 MACs (2–4× faster)
- Accuracy drop: typically <1% with QAT
- Enables deployment on Cortex-M0+ (no FPU)

Knowledge Distillation Effects

- Teacher: large DS-CNN-L (95.6% accuracy)
- Student: tiny DS-CNN-S trained on soft labels
- Accuracy recovery: +1.5–2% vs. training from scratch
- Soft labels carry inter-class similarity information

KWS Training & Deployment Pipeline



The pipeline separates training-time optimization (augmentation, quantization-aware training) from deployment-time execution (TFLM interpreter running on-device inference from microphone input). The exported INT8 TFLite model is the bridge between these two phases.



SECTION 12

Visual Wake Words

Camera-based person detection at microwatt power levels — enabling intelligent scene understanding on embedded vision systems.

Visual Wake Words (VWW)

Task Definition

VWW is the binary classification task of determining whether a specific object class (typically a person) is present in an image captured by an always-on camera. It is the visual analogue of keyword spotting — a low-power trigger for downstream processing.

Deployment Context

- Smart doorbells and security cameras
- Occupancy detection in smart buildings
- Wake trigger for high-power vision pipelines
- Industrial zone monitoring systems

Person Detection & COCO Dataset

COCO Dataset

330K images, 80 object categories. The "person" category contains 262K labeled instances — the largest in the dataset, providing rich training signal for VWW.

VWW Subset

115,000 training images derived from COCO. Balanced 50/50 person vs. non-person. Images resized to 96×96 or 96×96×1 (grayscale) for MCU deployment.

Performance Target

≥80% accuracy on VWW test set while fitting within 250 KB flash and 100 KB SRAM — the reference MCU constraints for MLPerf Tiny.

MobileNet Architecture

MobileNet is the standard backbone for visual TinyML tasks. It systematically replaces all standard 3×3 convolutions with depthwise separable convolutions, reducing FLOPs by $\sim 8-9\times$ and parameters by a comparable factor – with minimal accuracy loss on visual benchmarks.

01

Input Image

$96\times 96\times 3$ RGB or $96\times 96\times 1$ grayscale tensor

03

13 DS Blocks

Depthwise + Pointwise separable convolution stacks with stride control

02

Entry Conv

Standard 3×3 Conv $\rightarrow$ BN $\rightarrow$ ReLU6

04

Global AvgPool $\rightarrow$ FC $\rightarrow$ Sigmoid

Collapse spatial dimensions $\rightarrow$ binary classification output

Depthwise Separable Convolutions

Standard Conv (Expensive)

A $3 \times 3 \times C_{in} \times C_{out}$ kernel applies spatial and channel mixing simultaneously.

Cost: $Dk^2 \times C_{in} \times C_{out} \times Df^2$ MACs

Depthwise Separable (Efficient)

Step 1 — Depthwise: $3 \times 3 \times 1$ filter per channel (spatial only)

Step 2 — Pointwise: $1 \times 1 \times C_{in} \times C_{out}$ (channel mixing)

Reduction: $\sim 8-9\times$ fewer MACs for $C_{out}=128$

✔ MobileNetV1 on VWW achieves 85% accuracy with only 300K parameters — ideal for Cortex-M4 deployment.

MobileNet Compression Strategies

1

Width Multiplier α

Scale all channel counts by $\alpha \in (0,1]$. $\alpha=0.25$ reduces parameters by $\sim 16\times$ with $\sim 8\%$ accuracy drop.

2

Resolution Multiplier ρ

Reduce input resolution (e.g., $224 \rightarrow 96$). Reduces FLOPs quadratically with linear accuracy cost.

3

INT8 Quantization

Post-training or QAT converts float32 weights to 8-bit integers. $4\times$ size reduction, $2-4\times$ faster inference.

4

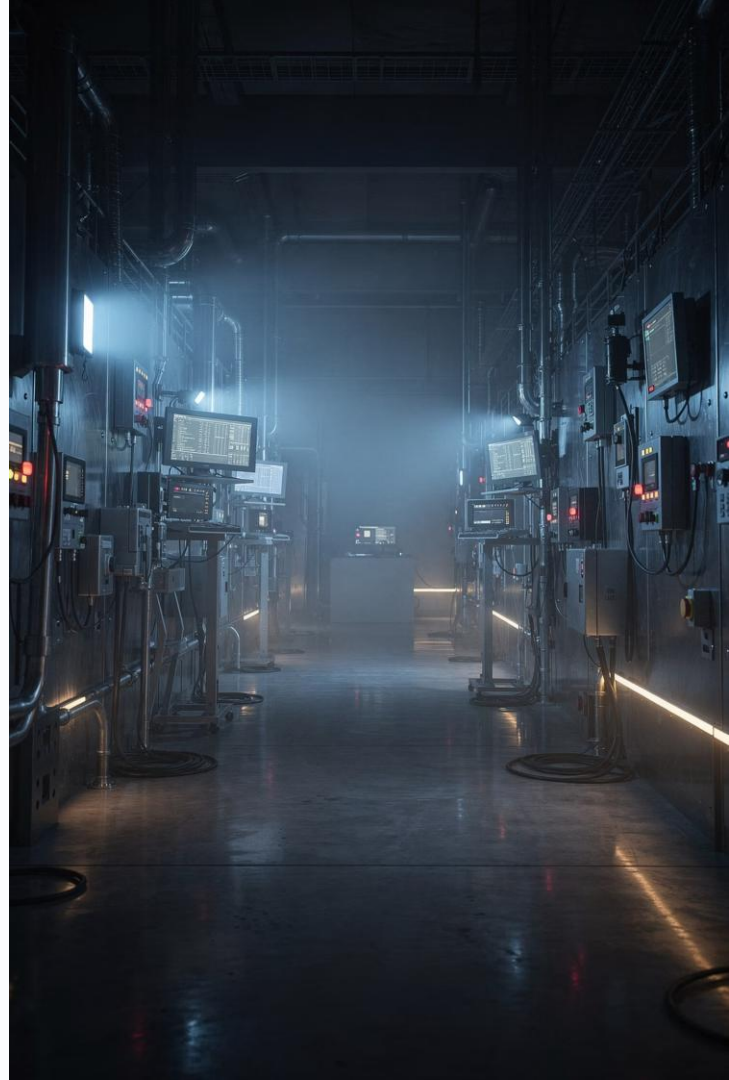
Pruning

Remove low-magnitude filters. Up to 50% sparsity with $<1\%$ accuracy drop after fine-tuning.

SECTION 13

Anomaly Detection

Identifying abnormal sensor patterns without labeled examples of failure — the core challenge of predictive maintenance at the edge.



What Is Anomaly Detection?

Core Concept

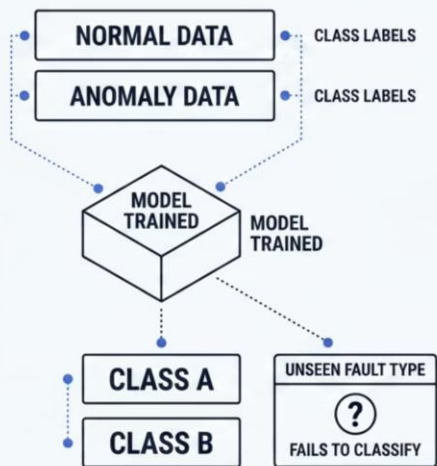
Anomaly detection identifies data points that deviate significantly from a learned model of "normal" behavior. Unlike classification, it requires **no labeled anomaly examples** during training — the model learns only from normal operating data.

TinyML Applications

- Bearing fault detection via vibration (IMU)
- Motor sound anomaly detection via microphone
- HVAC airflow degradation monitoring
- Structural health monitoring in bridges

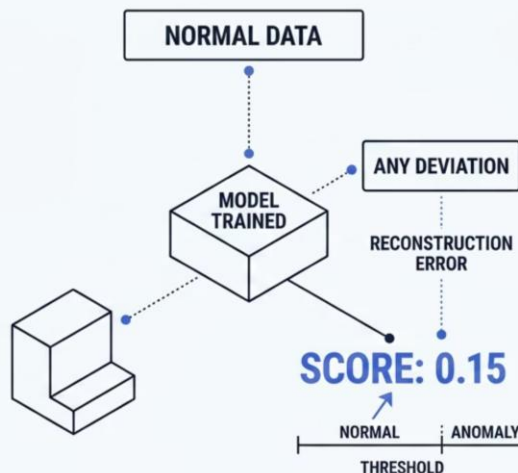
Classification vs. Anomaly Detection

CLASSIFICATION



CLASSIFICATION

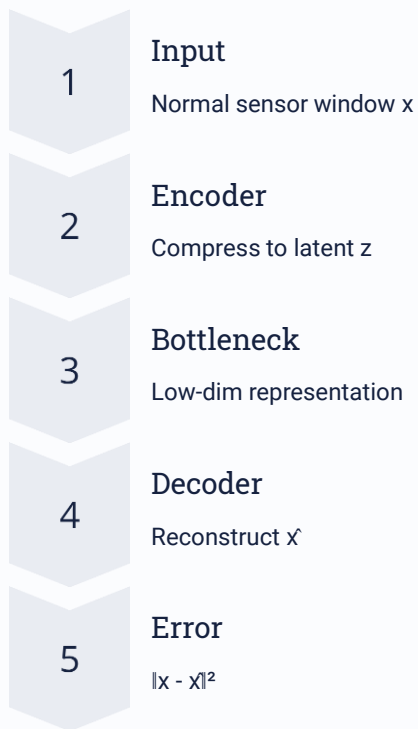
ANOMALY DETECTION



ANOMALY DETECTION

Autoencoders for Anomaly Detection

An autoencoder is a neural network trained to reconstruct its input through a compressed bottleneck representation. Trained only on normal data, it learns to reconstruct normal patterns accurately – but produces high reconstruction error on anomalous inputs it has never seen.



Reconstruction Error & Thresholding

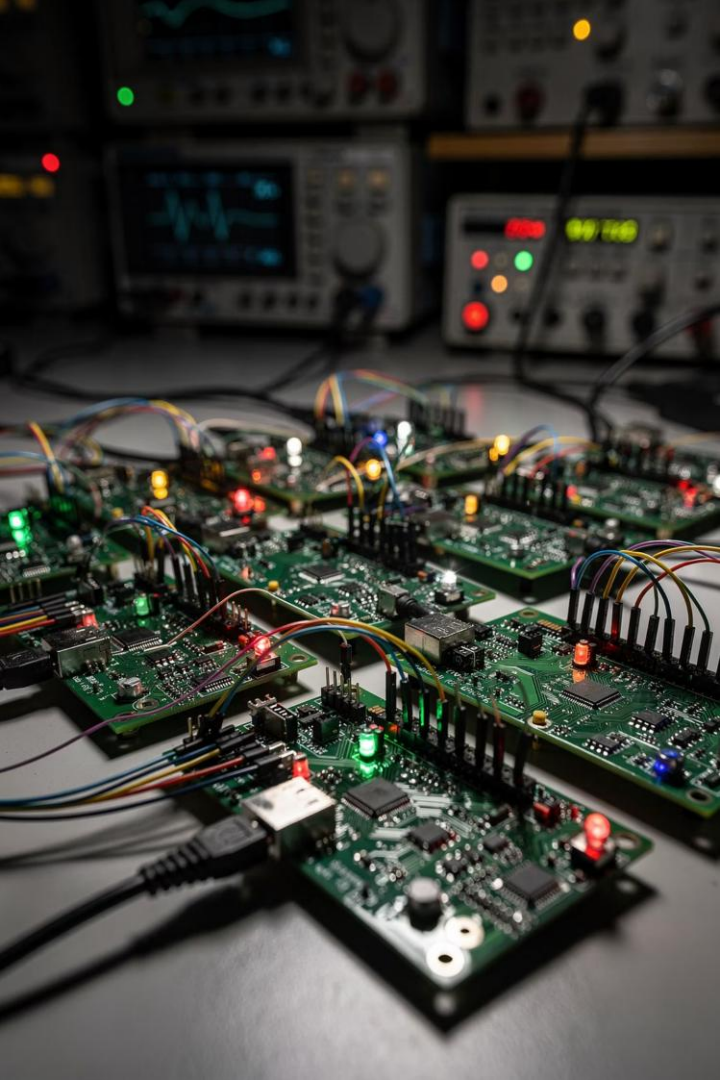
How Thresholding Works

- Compute reconstruction error on a held-out normal validation set
- Set threshold τ at the 95th or 99th percentile of normal errors
- At inference: if error $> \tau \rightarrow$ flag as anomaly
- Threshold is a deployable scalar constant — zero extra flash cost

Deployment on MCU

- Autoencoder quantized to INT8 via TFLM
- Pruning reduces encoder depth by 50–70%
- Typical MCU cost: <50 KB flash, <20 KB SRAM
- Inference triggered on fixed 128-sample windows

 Pruning and INT8 quantization together can reduce autoencoder flash footprint by 8× with <2% AUC degradation on MIMII benchmark data.



SECTION 14

Multimodal TinyML

Fusing heterogeneous sensor streams on a single embedded processor to make more robust, context-aware decisions at the edge.

Sensor Fusion in TinyML

Single-modality models are brittle — a microphone-based model fails in noisy environments; a camera-based model fails in darkness. Sensor fusion combines complementary modalities so that each compensates for the other's weaknesses, improving system reliability without requiring a more powerful processor.



Audio + Vision

Person detection confirmed by both camera presence and voice activity. Reduces false triggers in smart home scenarios.



Audio + IMU

Combine acoustic anomaly score with vibration signature. Stronger anomaly detection for rotating machinery.

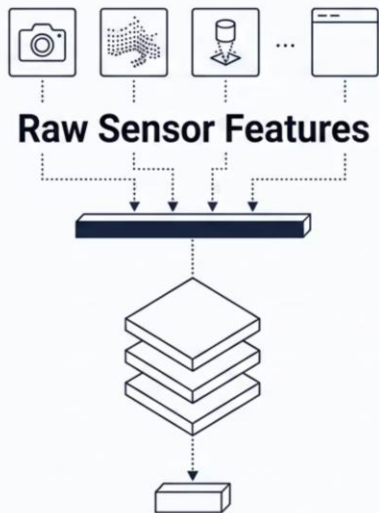


Industrial Multimodal

Temperature + current + vibration fusion for comprehensive predictive maintenance on production equipment.

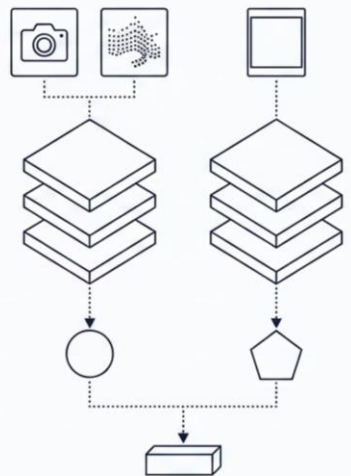
Multimodal Fusion Strategies

Early Fusion



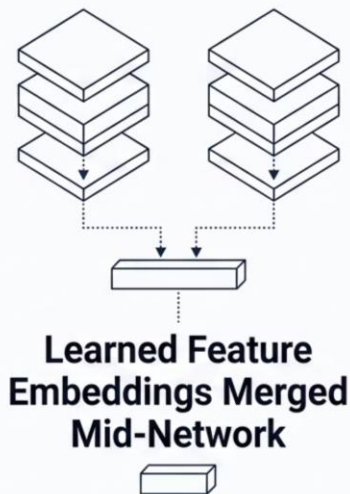
Before model

Late Fusion



Voting or Averaging
Predictions

Intermediate Fusion



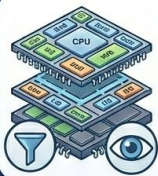
Shared Representation
Learning

Late fusion is most MCU-friendly: each modality runs as an independent quantized model, and their output scores are combined with a simple weighted average — requiring no extra parameters or memory overhead for the fusion step itself.



Responsible AI for TinyML

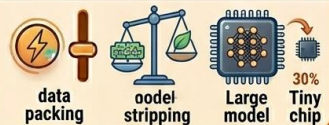
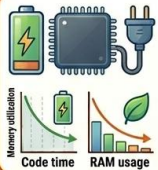
Data Privacy and Bias



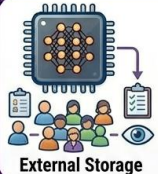
Transparency and Robustness



Sustainability and Resource Efficiency



Accountability and Safety



SECTION 15

Responsible AI for TinyML

Dataset Bias & Fairness

Real-World TinyML Fairness Failures

- **Wake words:** "Hey Siri" false rejection rate 30% higher for non-native English speakers in early testing
- **Gesture recognition:** IMU-based models trained on dominant hand perform poorly for left-handed users
- **Face detection:** darker skin tones underrepresented in training data, triggering disparate camera activation rates

Fairness Metrics to Track

- Disaggregated accuracy by demographic group
- Equalized odds across protected attributes
- False positive / false negative rate parity
- Dataset representation audits pre-training

Model Cards & Deployment Risks

Model Cards

Standardized documentation (Google, 2019) covering: intended use, evaluation data, disaggregated performance metrics, known limitations, and recommended use constraints.

Deployment Risks

TinyML devices are difficult to patch post-deployment. A biased or insecure model embedded in hardware may operate for years without correction — amplifying harms at scale.

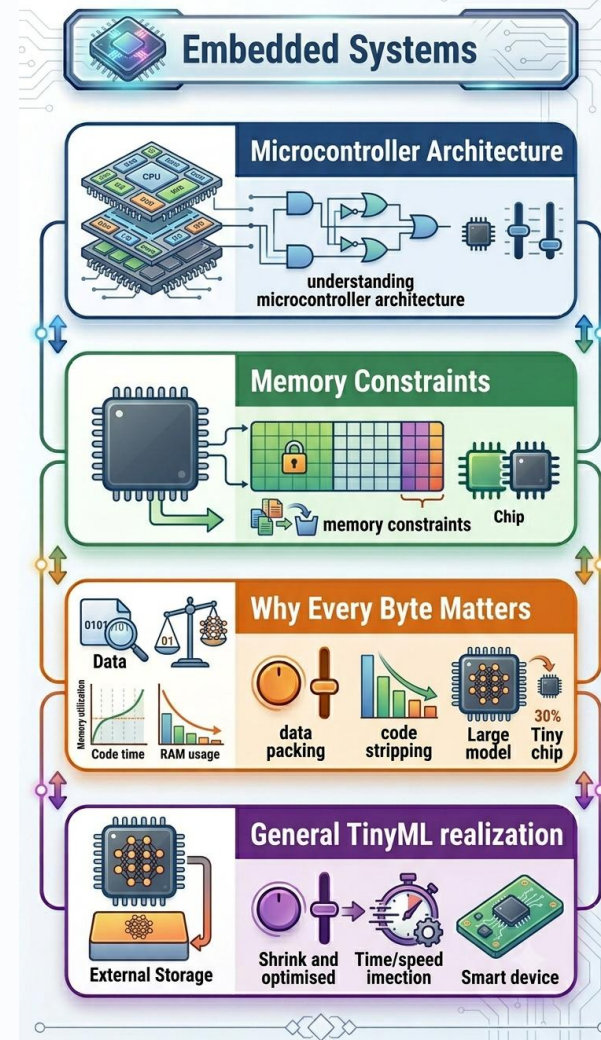
Monitoring

Lightweight on-device anomaly detection on the inference distribution can flag data drift before performance degrades. Log anonymized statistics, not raw sensor data.

SECTION 16

Embedded Systems

The physical foundation of TinyML — understanding microcontroller architecture, memory constraints, and why every byte matters.



MCU Fundamentals

What Is an MCU?

A microcontroller unit (MCU) integrates a CPU core, flash memory, SRAM, and peripherals (ADC, GPIO, UART, SPI, I2C) on a single silicon die. Unlike a microprocessor, it is designed for embedded, real-time control – not general-purpose computing.

TinyML-Relevant Parameters

- **Core:** ARM Cortex-M0/M4/M7 or RISC-V
- **Clock:** 16 MHz – 480 MHz
- **Flash:** 32 KB – 16 MB (stores model + code)
- **SRAM:** 4 KB – 1 MB (activations + Tensor Arena)
- **Power:** 10 μ W (sleep) – 100 mW (active inference)

Arduino Nano 33 BLE Sense

Processor

nRF52840 ARM Cortex-M4F @ 64 MHz with hardware FPU

Memory

1 MB Flash / 256 KB SRAM — tight but sufficient for INT8 KWS and VWW models

Sensors

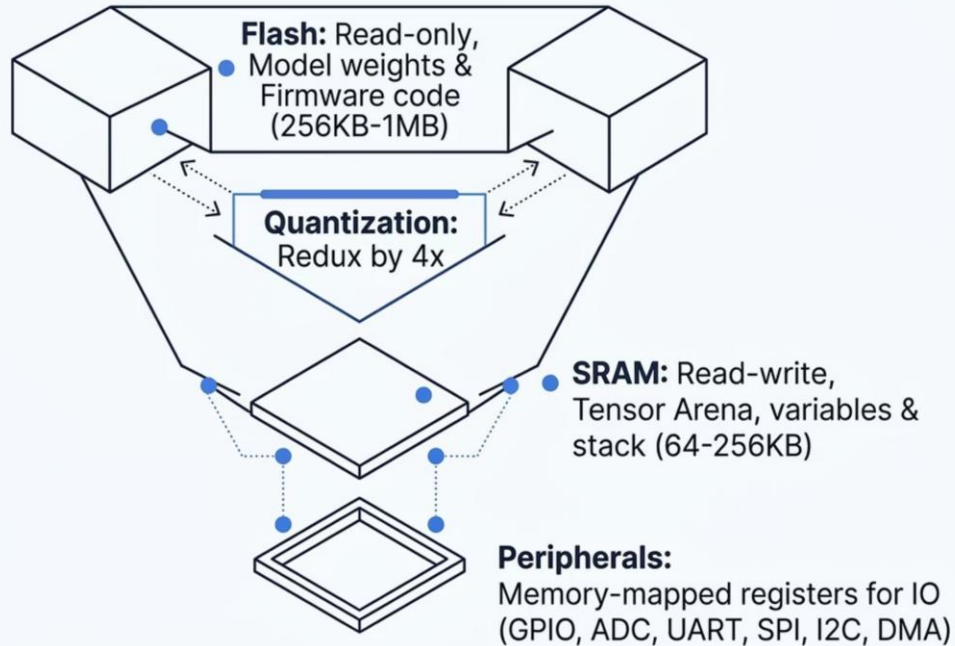
9-axis IMU, PDM microphone, temperature, humidity, pressure, color, proximity — all on one board

Connectivity

Bluetooth 5 BLE for wireless data logging and OTA model updates during prototyping

 The Arduino Nano 33 BLE Sense is the reference platform for the Harvard CS249r TinyML course and MLPerf Tiny submissions.

Flash vs. SRAM: Memory Architecture



Quantization is critical for memory-constrained MCUs: converting float32 weights to INT8 reduces the model's flash footprint by 4x. A 320 KB float32 model becomes 80 KB INT8 — fitting comfortably in the Nano 33's 1 MB flash alongside the TFLM runtime and application code.

Tensor Arena

What Is the Tensor Arena?

The Tensor Arena is a statically allocated block of SRAM reserved exclusively for TFLM to store intermediate activation tensors during inference. Its size must be determined at compile time — it cannot grow dynamically on a bare-metal MCU.

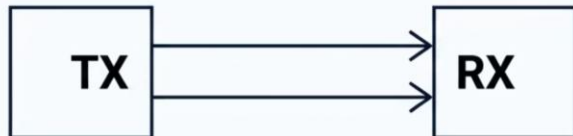
```
static uint8_t tensor_arena[50 * 1024]; // 50 KB
MicroInterpreter interpreter(model, resolver,
    tensor_arena, sizeof(tensor_arena));
```

Sizing the Arena

- Start large (e.g., 100 KB), run inference, check reported arena usage
- Reduce to reported size + 10% safety margin
- Arena size = peak activation memory, not total model size
- Rule of thumb: arena $\approx 2\text{--}3\times$ the largest layer's activation tensor

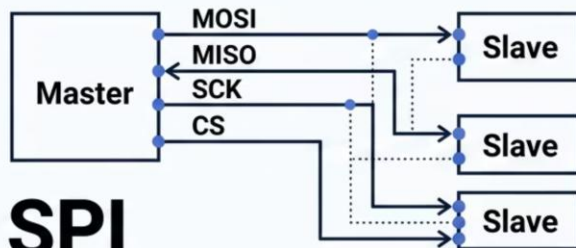
Communication Protocols

Embedded TinyML systems communicate with sensors, displays, and host systems via standardized serial protocols. Selecting the right protocol depends on speed requirements, wire count constraints, and the number of devices on the bus.



UART

2 wires: TX, RX.
Simple stream.

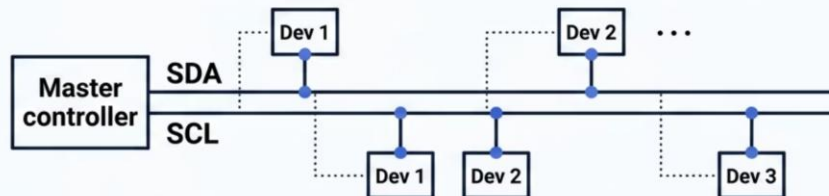


SPI

4 wires: MOSI, MISO, SCK, CS.
Fast full-duplex.

I2C

2 wires: SDA, SCL. Multi-device bus.



TFLM

Remove of external software



Code Stripping



Data Packing



File size

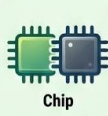
No OS



Direct memory access



Manual reuse



Chip

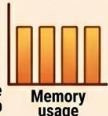
No Dynamic Memory Allocation



Total control



Dynamic voltage placement/ heap



Memory usage

No C++ Standard Library



Direct linking application



Only essential functions



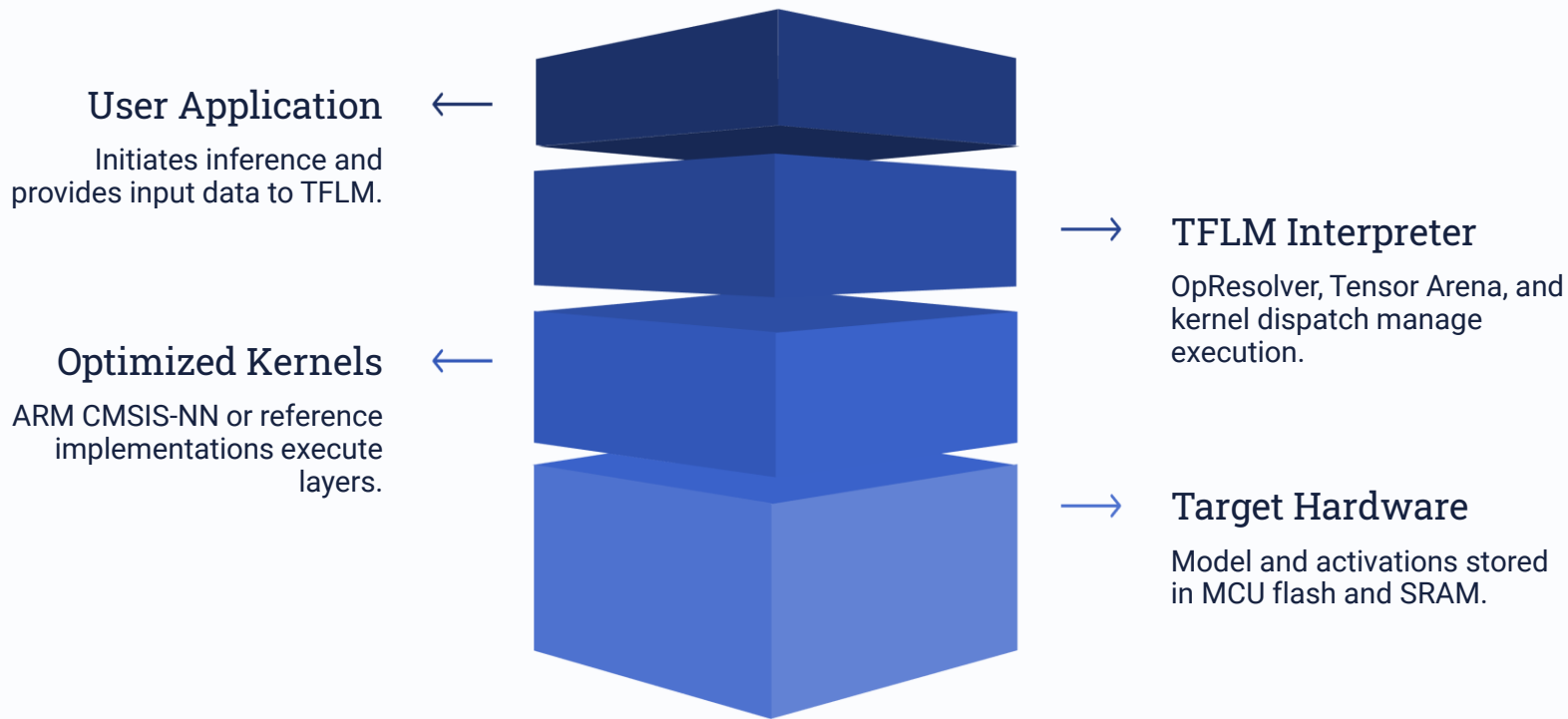
Direct function execution

SECTION 18

TensorFlow Lite Micro (TFLM)

TFLM is the bare-metal inference runtime that brings trained TensorFlow models to microcontrollers — no OS, no dynamic memory allocation, no standard C++ library dependencies.

TFLM Architecture



The TFLM Interpreter reads a FlatBuffer-serialized model from flash, allocates activations in the Tensor Arena, and dispatches each layer's kernel to either a hardware-optimized implementation (ARM CMSIS-NN for Cortex-M) or a portable reference kernel. No heap allocation occurs during inference.

TFLM Key Components



Interpreter

Orchestrates inference: resolves ops, allocates tensors in the arena, executes kernels in topological order. Statically constructed at startup.



OpResolver

Registers only the ops needed by your model. Reduces binary size by excluding unused kernels — critical for flash-constrained devices.



Tensor Arena

Static SRAM buffer for all intermediate activations. Zero dynamic allocation. Size is your primary SRAM budget lever.



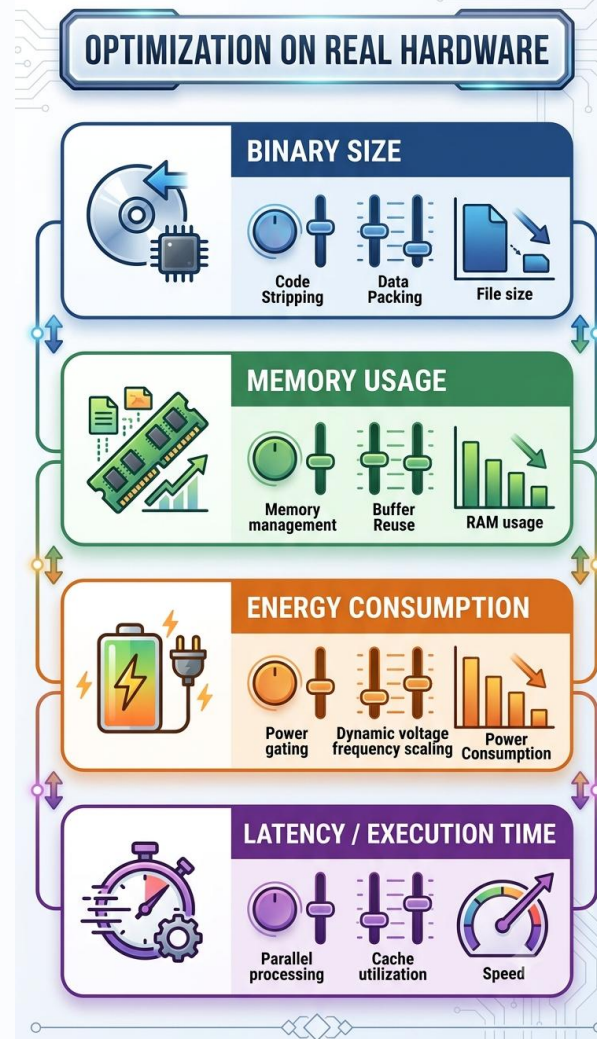
CMSIS-NN Kernels

ARM's hand-optimized INT8 kernel library. Uses SIMD DSP instructions on Cortex-M4/M7 to deliver 4–8× speedup vs. reference kernels.

SECTION 19

Optimization on Real Hardware

Optimization for embedded deployment spans four dimensions: binary size, memory, energy, and latency. Each dimension has distinct levers — and all four interact. Optimizing one often improves the others.



Binary Size, Memory & Latency Optimization

Binary Size

- Use MicroMutableOpResolver (register only needed ops)
- INT8 quantization: 4× model size reduction
- Link-time optimization (LTO) removes dead code
- Pruning: 50% sparsity → ~2× size reduction

Memory & Latency

- Size Tensor Arena to minimum required allocation
- Enable ARM CMSIS-NN for INT8 SIMD acceleration
- Distillation: smaller student model, same accuracy
- Maximize MCU clock during inference windows

- ✓ Energy optimization: gate MCU peripherals between inference cycles. Sleep modes between inferences can reduce average power by 100× on duty-cycled systems.

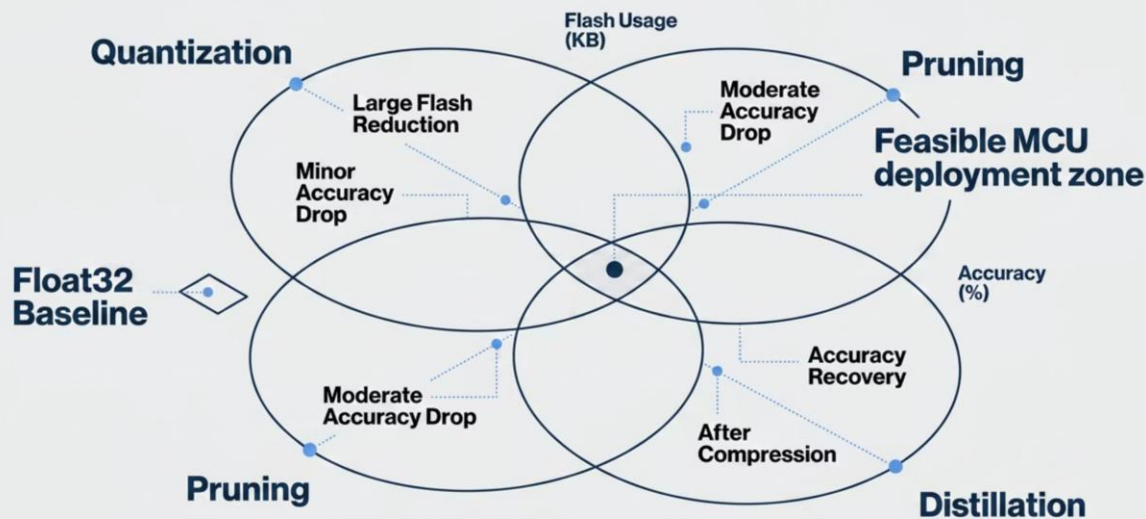
Optimization Benchmark Results

The following table shows measured effects of optimization techniques on a DS-CNN KWS model deployed on Arduino Nano 33 BLE Sense (Cortex-M4, 64 MHz).

Optimization	Flash (KB)	SRAM (KB)	Latency (ms)	Accuracy
Float32 Baseline	320	96	210	95.4%
INT8 Quantization	80	30	62	94.8%
+ 50% Pruning	42	28	38	94.1%
+ Distillation	42	28	38	95.2%

Accuracy vs. Resource Trade-offs

Every TinyML design decision involves a Pareto trade-off: squeezing resources costs accuracy. Understanding this frontier is essential for selecting the right operating point for a given application's constraints.



Optimization Impact Summary

4×

Flash Reduction

INT8 quantization vs. float32 baseline

3.4×

Latency Speedup

INT8 CMSIS-NN kernels vs. float32 reference

50%

Additional Size Cut

Structured pruning on top of quantization

+1.1%

Accuracy Recovery

Knowledge distillation applied post-pruning

MLPerf Tiny Benchmark Suite

MLPerf Tiny is the industry-standard benchmark suite for evaluating TinyML systems across four tasks. It standardizes models, datasets, and evaluation protocols so that MCU vendors and researchers can make fair, reproducible comparisons.

KWS

DS-CNN on Google Speech Commands. Metric: Top-1 accuracy. Target: $\geq 90\%$

VWW

MobileNetV1 on VWW dataset. Metric: Binary accuracy. Target: $\geq 80\%$

Image Class.

ResNet-8 on CIFAR-10. Metric: Top-1 accuracy. Target: $\geq 85\%$

Anomaly Det.

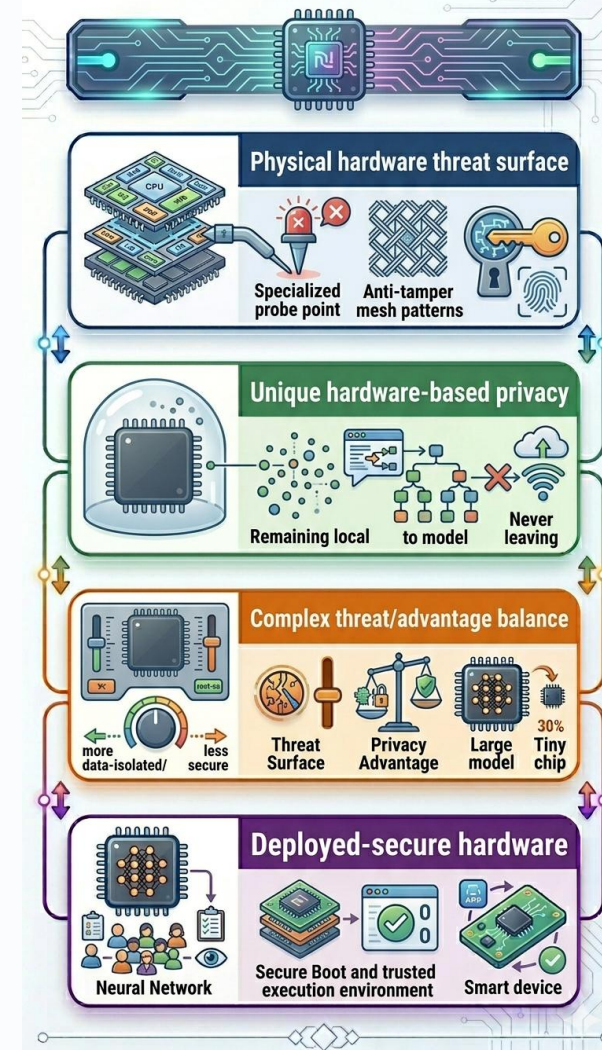
Autoencoder on MIMII dataset. Metric: AUC-ROC. Target: ≥ 0.85

 MLPerf Tiny v1.0 (2021) received submissions from ARM, ST Microelectronics, Ambiq, and academic teams — establishing a global leaderboard for edge inference efficiency.

SECTION 22

Security & Privacy

Deploying ML models on physical hardware creates unique threat surfaces — and unique privacy advantages. Both must be understood.



TinyML Security Threats

Model Theft

An adversary with physical access to an MCU can extract the model weights from flash memory using JTAG/SWD debug interfaces — then reverse-engineer or clone the proprietary model. Mitigations: readback protection bits, encrypted flash, TrustZone secure storage.

Adversarial Attacks

Small, human-imperceptible perturbations to input signals can cause confident misclassification. Physical adversarial examples (printed patches, acoustic tones) have been demonstrated against deployed TinyML vision and audio models in the real world.

Privacy Advantages of TinyML



No Cloud Transmission

Raw audio, images, and biometric data never leave the device. Only a binary decision or anonymized aggregate is transmitted — dramatically reducing data exposure risk.



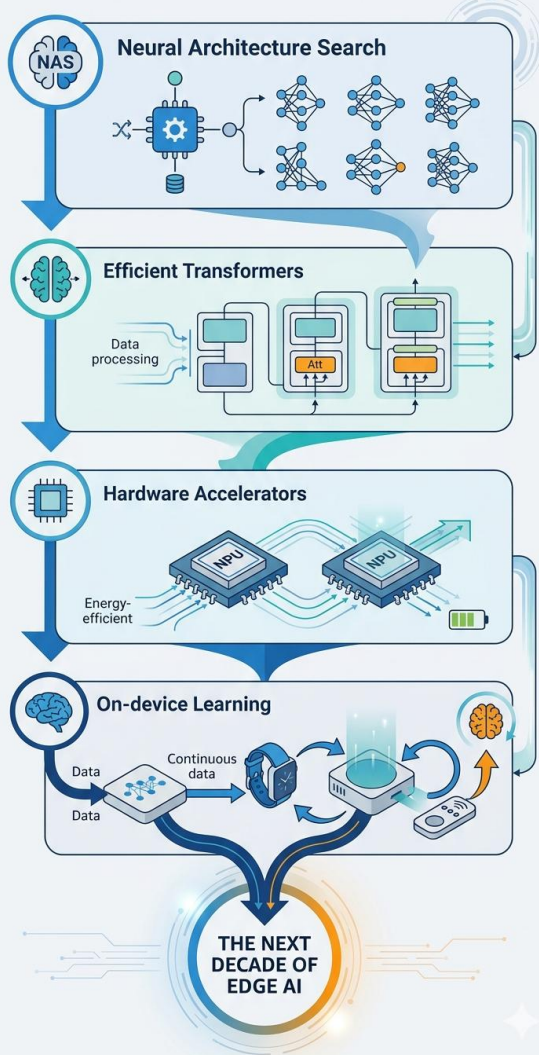
GDPR / HIPAA Alignment

On-device processing sidesteps many data residency and consent requirements because no personal data is stored or transmitted to third-party servers.



Federated Learning

Model updates computed locally and only gradient updates (not raw data) are shared. Enables privacy-preserving collaborative learning across millions of edge devices.



SECTION 23

The Future of TinyML

Neural Architecture Search, efficient transformers, hardware accelerators, and on-device learning are converging to define the next decade of edge AI.

Hardware-Aware NAS & Efficient Architectures

TinyNAS / Hardware-Aware NAS

Automatically searches the architecture space under explicit MCU constraints (flash, SRAM, latency). Finds Pareto-optimal models unreachable by manual design. Used in MCUNet, TinyEngine.

EfficientNet Lite / MobileNetV4

Systematically compound-scaled architectures optimized for INT8 inference on edge hardware. MobileNetV4 introduces Universal Inverted Bottleneck and attention mechanisms while remaining MCU-deployable.

TinyFormer / Tiny Transformers

Transformer architectures adapted for MCUs via linear attention, patch embedding compression, and aggressive quantization. Approaching CNN accuracy on audio and vision tasks at comparable model sizes.

Edge AI Accelerators & On-Device Learning

NPU & Edge Accelerators

- Dedicated Neural Processing Units (NPUs) on MCUs: Ambiq Apollo, STM32N6, Nordic nPM1300 with ML extensions
- 10–100× inference speedup vs. general-purpose Cortex-M
- Hardware support for INT4/INT8 MAC arrays and sparse operations
- Spiking Neural Networks (SNNs): event-driven, ultra-low energy, neuromorphic chips (Intel Loihi, BrainScaleS)

On-Device & Federated Learning

- Federated Learning: gradient updates aggregated across device fleets — raw data never shared
- On-device fine-tuning: personalize KWS models to individual speaker voices post-deployment
- Binary Neural Networks (BNNs): weights $\in \{-1, +1\}$, XNOR-popcount operations replace MACs — extreme compression for inference-only tasks

Course Project: TinyML Robot Navigation

The capstone project integrates all course concepts: students build a sensor-fused navigation system on an Arduino Nano 33 BLE Sense, fusing IMU and ultrasonic sensor data through a quantized INT8 model to detect obstacles and navigate autonomously – all within a 50 KB flash budget and sub-100 ms latency constraint.

01

Data Collection

Log IMU + ultrasonic readings in real navigation scenarios

02

Train & Quantize

Train obstacle classifier, apply INT8 QAT, export to TFLite

03

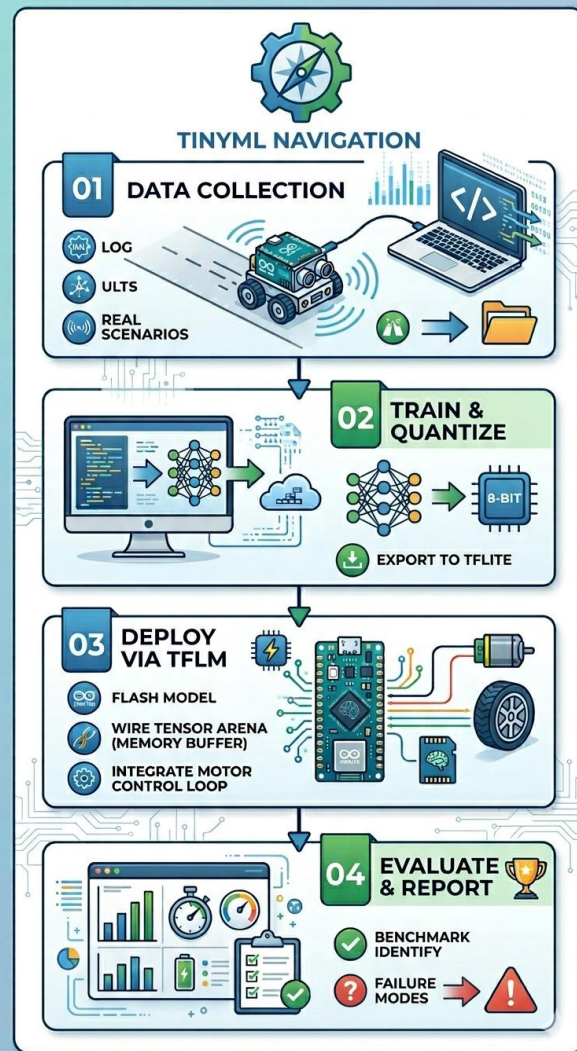
Deploy via TFLM

Flash model, wire Tensor Arena, integrate motor control loop

04

Evaluate & Report

Benchmark accuracy, latency, power, and failure modes



The Anatomy of a Successful TinyML System

Good Data Curated, balanced, augmented, ethically governed datasets	Efficient Architecture DS-CNN, MobileNet, TinyFormer — hardware-aware design
Distillation Transfer teacher knowledge to a compact student model	Pruning Remove redundant weights, preserve accuracy
Quantization INT8 inference — 4× smaller, 4× faster, fraction of the power	TFLM Deploy Interpreter + OpResolver + Tensor Arena on bare metal MCU

"The future of AI is not only larger models in the cloud. The future is also **smaller, faster, cheaper, and more intelligent systems at the edge.**"